PROJECT CHARTER
TEAM Samson – SQUAD Job Applicant

GENERAL PROJECT INFORMATION

PROJECT NAME		
DEVision - Job Applicant Subsystem		
SQUAD NAME	TEAM CODE (Job Applicant or Job Manager)	PROJECT MANAGER (Your team's representative)
Samson	Job Applicant	Kiet Hong Thieu
TEAM MEMBERS		
Huy Truong Phuoc — S3891442@rmit.edu.vn Kiet Hong Thieu — S3993986@rmit.edu.vn Hieu Ngo Quang — S3820591@rmit.edu.vn Dat Tran Xuan Hoang — S3651550@rmit.edu.vn Quang Tran — S3976073@rmit.edu.vn		
STAKEHOLDERS (If NO client or company involves in the project, state your lecturer's full name)		
Tri Huynh		
EXPECTED START DATE	EXPECTED DATE OF COMPLETION	DATE OF DOCUMENT
07/11/2025	13/01/2026	07/11/2025

PROJECT DETAILS

VISION	The following questions serve as hints to your vision definition: - What system features will you develop? What benefits do you want to provide for your end users? - What are your target frontend and backend architectures? - How will you deploy your system? - What are the desired properties (maintainable, scalable, reusable) of your system design? - The goal of this system is to provide a modern, secure, and reusable User Management module where users can register, activate their account via email link, log in, recover password, and update personal information. - For Admin, the system supports user role management and deleting the Job post.

- HR account can successfully post the JD on the platform with full tag of requirements for the User to find if they are suitable for the Job. Maybe show the location of the company on the map. HR has the right to CRUD their Job Post.

- User Profile have full tag of skill they have. Having clip or video of the past project to prototype to the HR or maybe they can upload their CV on the platform. User will success update they profile (change password, add more tag of skills…) and can delete (CRUD)

- Based on the User skill tag, the homepage can show some jobs suggestion based on it.

- There will be a search bar for users to search for the job they want.

- The system will be developed with **TypeScript** on both frontend and backend to improve type-safety, reduce runtime errors, and make code easier to maintain.
  The frontend will be implemented using **React + Tailwind CSS** (and Bootstrap UI components where needed) to ensure a fast development workflow with responsive UI.
- The backend will use Modular Monolith architecture and **Node.js + Express + MongoDB**, also written in TypeScript.
- We will integrate **Apache Kafka** to decouple asynchronous processes such as sending activation emails, logging audit trails, and event notifications. This improves scalability and reliability when system load increases.
- The deployment target will be cloud platforms such as Vercel (frontend) and Railway / Render (backend). Kafka can be deployed using a managed service or Docker container depending on resource and hosting availability.
- We aim for a system that is:
- **Maintainable** → coding standards with TypeScript, layered architecture (Controller → Service → Repository)
- **Scalable** → Kafka event-driven handling enables future modules to subscribe to events without changing core logic
- **Reusable** → The User module can be reused for other projects that require authentication
- **Modern UI** → Tailwind + Bootstrap gives clean, responsive interface with rapid styling
- In summary, this project demonstrates modern web architecture using TypeScript end-to-end, async event processing with Kafka, and cloud deployment readiness.

SUCCESS METRICS	- How do you measure the quality of your team's delivery in achieving the above visions?
	- First, the User Stories must be completed correctly and work as expected (target >= 95%)
	- The FE and BE code should have no serious TypeScript or ESLint errors. The UI must be responsive and display correctly on different screen sizes.
	- For performance, the backend API should respond fast (avg <300ms), Kafka must run stably and process events successfully (>99%)
	- Job Applicant module can be integrated smoothly with the Job Management system developed by another team, without requiring heavy refactoring.
	  - We measure our code quality with some metrics:
	  Complexity: How many statements in a method. The higher the score, the more complex the code.
	  Code Documentation: Measure the quality of the code and how easy it is for others to understand and use. The higher the score, the easier the code is to understand.

RISKS	- What are the impediments to the development of your project? - Unfamiliarity: lack of Kafka/Docker experience. - Deployment issue: application crashes in production. - Dependencies issues: third-party libraries having updates or being deprecated makes conflict in production. - Performance issue: application becomes slower, slow database queries, poor caching. - API misalignment: may conflict with JM services. - Environment drift: may work or not work on some specific computers.
MILESTONES	- **Sprint 1 (W2-W3):** **GitHub Project Repository Setup:** - Create repository and define branching methods. - All members start performing necessary and frequent commits and pushes to ensure active participation from everyone. **Data modelling:** - Identify all entities required by the Job Applicant subsystem (Applicant, Education, Experience, Skills, Subscription, Application, etc.) - Draft and complete conceptual ER models using a modeling tool. - Validate relationships against functional requirements. **Early Technology Decisions:** - Apply React + Tailwind CSS (optional Bootstrap components where needed) for frontend development. - Node.js + Express + MongoDB written in TypeScript for backend along with Modular Monolith architecture.

	Documentation: - Write Data Model explanation and justification (discuss advantages & drawbacks per requirements whether met or not). - **Sprint 2 (W4-W5):** **Database and system architecture design:** - Fine-tune architectural system designs which satisfy the requirements stated up to at least Medium level with a reasonable attempt at the Ultimo level, complying with all the features listed within the Medium level. - Frontend architecture diagram showing componentized designs (pages + UI components, etc.) - Backend architecture diagram showcasing: o N-tier + Modular Monolith. o Complete demonstration of help classes and utility configurations (e.g. Token Generator) - Discuss and brainstorm the expected system's maintainability, scalability, security, and overall performance. - Finalize and verify all the necessary diagrams and the 30-page report no later than **Thursday (27/11/2025)**. - Aim to s**ubmit within Thursday (27/11/2025)** if possible, otherwise make final adjustments and verifications and submit within the next morning.
TEAM RULES	- State **communication frequency and model**, e.g., daily team meeting online; daily progress synchronization on the team's channel on Discord. - State project management board practices: update status on Jira once the tasks are done, always assign a member and due date of a task, etc. - State **tolerable communication delay policy**, e.g., members must respond to the team's request or message within 24 or 48 hours, except non-working days. - State first-step countermeasure if someone violates the rules or deadlines, e.g., assign a small task to be done within 2 working days.

APPROVAL SIGNATURES

By signing below, each member commits to the objectives, rules, and guidelines outlined in this Project Charter.

Tran Xuân Hoang Dat	Tran Quang	Truong Phuoc Huy	Ngo Quang Hieu	Hong Thieu Kiet